READING SHEET

Optimisation & Environment

Module 3 · Frontend: React & Next.js · Day 43 · Week 9

PROGRAM	WORK TYPE	EST. TIME
CodeOps · Full Stack	**Revision + mini-project**	**2–3 hours**

Tip How to use this sheet

A correct application that takes six seconds to appear on a phone is not finished. This sheet covers the image and font components that remove most of that time, the three numbers that describe how a page feels, and the configuration that lets one codebase run on your laptop, in preview and in production. Take a Lighthouse measurement before you change anything — the rest of the sheet is only useful against a starting point.

Part 1 · Images

Images are almost always the heaviest thing on a page. On a phone on mobile data in Addis, one unoptimised photograph of a dish can cost more than every line of JavaScript you have written this module.

PROBLEM	WHAT next/image DOES
A 4 MB photograph	Resizes it to what the screen needs
JPEG on a modern phone	Serves WebP or AVIF instead
One size for every device	Generates a srcset automatically
Everything loads at once	Lazy-loads anything below the fold
The page jumps as it loads	Reserves the space in advance

app/menu/DishCard.jsx

```
import Image from "next/image";

<Image src="/dishes/doro-wat.jpg" alt="Doro Wat with injera"
       width={400} height={300}
       sizes="(max-width: 768px) 100vw, 400px" />
```

width and **height** are the aspect ratio, not the display size — they exist so the browser can reserve the right space before the image arrives, which is what stops the layout jumping. **sizes** is not optional either: without it the browser assumes full width and downloads the largest version, so a card three across costs three times the bytes it needs.

priority, fill and remote images

```
                                                    the three options you will actually use
<Image src="/hero.jpg" alt="" priority ... />   // the one big image
<Image src={url} alt="" fill ... />             // fills its container

// next.config.mjs
images: { remotePatterns: [{ protocol: "https",
                             hostname: "cdn.addiseats.et" }] }
```

> **Warning priority on one image only**
> It disables lazy loading and preloads the file — use it on the largest image visible without scrolling and nowhere else, or you are competing with yourself for bandwidth. An unlisted remote host throws at build time, which is deliberate: it stops your optimiser being used to resize other people's images.

Getting images right at the source

DO	INSTEAD OF
Upload at roughly display size	A 6000px camera original
Photographs as JPEG or WebP	PNG for a photograph
Logos and icons as SVG	A 200 kB PNG logo
Meaningful alt text, empty for decoration	alt="image", or nothing at all

The component is not magic — it cannot make a badly chosen format small. And alt text has not stopped mattering since Module 2: a menu whose dishes have no alt text is unusable with a screen reader.

Part 2 · Fonts & Scripts

A font loaded the ordinary way costs a request to another company's server, a privacy question, and a flash of the wrong text. `next/font` removes all three at build time.

```
                                          app/layout.js — downloaded at build, served by you
import { Inter } from "next/font/google";
const inter = Inter({ subsets: ["latin"], display: "swap" });

<html className={inter.className}>...</html>
```

The file is fetched once when you build and then served from your own origin — one fewer connection to set up, and no third party learning who visits your site. `next/font/local` does the same for a font file you ship yourself, which matters if you need an Amharic typeface Google does not host.

Why fonts cause layout shift

WHAT HAPPENS	WHAT YOU SEE
Font still downloading	Text in a fallback, or invisible
Font arrives	Everything re-renders in the new face
The two faces differ in width	Lines re-wrap, the page jumps

WHAT HAPPENS	WHAT YOU SEE
display: "swap"	Text is readable immediately
Automatic fallback matching	The jump is small or invisible

Invisible text hides your content; an unmatched fallback makes the page jump. next/font computes a fallback with matching metrics so the swap barely moves anything. Specifying `subsets` trims the file to the characters you need.

Third-party scripts

```
                                                  next/script — say when it may run
import Script from "next/script";
<Script src="https://analytics.example/s.js"
        strategy="afterInteractive" />
```

STRATEGY	WHEN IT LOADS
`beforeInteractive`	Before hydration — rare, and blocking
`afterInteractive`	Once the page is usable (the default)
`lazyOnload`	During idle time — chat widgets, analytics

An analytics tag or a chat widget can easily outweigh your entire application. Every third-party script is bytes, a connection and a privacy decision — ask what it is for before adding it, not after the page gets slow.

The rest of the bundle

HABIT	WHY
Import only what you use	A whole date library for one format call
Prefer the platform	Intl handles dates and ETB natively
Keep heavy work server-side	Server components ship no JavaScript
Lazy-load rare components	The Day 34 dynamic import, still valid
Check the build output	First Load JS tells you immediately

```
                                                  ETB formatting, with no library at all
new Intl.NumberFormat("en-ET", { style: "currency",
  currency: "ETB" }).format(640);   // ETB 640.00
```

Before installing anything, check whether the browser already does it. Dates, currency, number formatting and relative times are all built in.

Part 3 · Measuring What Matters

Optimisation without measurement is decoration. Three numbers describe how a page feels, and one matters far more than the others on a slow connection.

METRIC	WHAT IT MEASURES	GOOD	USUALLY CAUSED BY
LCP	When the main content appears	Under 2.5s	A huge hero image
CLS	How much the layout jumps	Under 0.1	Unreserved images or fonts
INP	How fast it responds to a tap	Under 200ms	Too much JavaScript

On a mid-range phone on Ethiopian mobile data, **LCP** is almost always the problem and almost always an image — fix that before anything else. CLS is the cheapest to fix: dimensions on images and next/font handle most of it. INP is largely solved already by Day 38, because a server component has no JavaScript to block anything.

Measuring honestly

TOOL	WHAT IT TELLS YOU
Lighthouse, in Chrome	A score and a prioritised list of causes
Network tab, throttled	What a real slow connection feels like
The build output	JavaScript cost per route
Bundle analyzer	Which package made it that size
A real phone	The truth, when you can manage it

> **Warning Never measure the dev server**
> The development build is unminified and compiles routes on demand. Every measurement must come from npm run build followed by npm run start, or the numbers mean nothing. Throttle every time: on office broadband every application is fast — set the network to Slow 4G and the CPU to 4x slowdown and test the experience your users actually have.

A worked example

CHANGE	LCP	PAGE WEIGHT
Starting point	6.1s	3.4 MB
img → Image with sizes	2.9s	480 kB
priority on the hero	2.2s	480 kB
next/font, and menu on the server	1.7s	390 kB

The first change did most of it, because images were seven-eighths of the page. Everything after is real but small — which is the point: measure first, then fix the biggest thing. The network waterfall draws exactly this breakdown, and one glance tells you which change is worth an hour and which is worth a minute. Note that a slow database query delays the HTML itself, so Day 37’s caching and Day 38’s parallel awaits are performance work too.

Part 4 · Environment & Configuration

The same code runs on your laptop, on a preview deployment and in production, against different databases and different keys. Configuration is what makes that possible without editing anything.

FILE	PURPOSE
`.env`	Defaults for everyone, committed
`.env.local`	Your own secrets — never committed
`.env.development`	Development defaults, committed
`.env.production`	Production defaults, committed
`Platform settings`	Real production secrets, never in git

A file with `.local` in the name is yours alone; everything else in that list is shared configuration and must contain nothing you would mind a stranger reading. Put `.env*.local` in .gitignore, and commit a `.env.example` listing every variable with dummy values — it is how the next person knows what to set, and it is safe to publish.

Public and private, again

```
                                                    the prefix decides everything
DATABASE_URL=postgres://...        # server only
SESSION_SECRET=...                 # server only

NEXT_PUBLIC_MAP_KEY=pk_...         # in the bundle
NEXT_PUBLIC_SITE_URL=https://...   # in the bundle
```

A NEXT_PUBLIC variable is **substituted into the code when you build**. It is readable by anyone and cannot be changed without a new build — both facts surprise people. Validate your environment with a schema when the application boots: a clear error on the first line beats a mysterious undefined three screens later.

Three environments

	DEVELOPMENT	PREVIEW	PRODUCTION
Runs on	Your laptop	Every pull request	The live site
Database	Local or a dev copy	A seeded test database	The real one
Payment keys	Test keys	Test keys	Live keys

A preview deployment per pull request means every change is reviewed as a running application rather than as a diff — a production build, on production infrastructure, with safe data. That makes it the right place to run Lighthouse and to hand a link to someone who is not a developer. Day 48 sets this up on Vercel.

next.config

```
                                                    next.config.mjs — what you will actually set
const nextConfig = {
  images: { remotePatterns: [...] },
  async redirects() { return [...]; },
  async headers()   { return [...]; }
};
```

Redirects for old URLs and security headers belong here rather than in middleware — they are static rules, and keeping them in config keeps middleware fast.

Assigned Reading

Required

1. **nextjs.org — Image optimization** What / why: the component, sizing, priority, fill and remote patterns.
2. **web.dev — Core Web Vitals** What / why: LCP, CLS and INP defined, with the usual causes of each.

Optional — go deeper

- **Font optimization** — Google and local fonts, subsets, and how layout shift is avoided.
- **Environment variables** — file order, NEXT_PUBLIC, and build-time versus runtime values.

Exercises

Measure first. Every step after that is judged against the starting numbers.

1. Run Lighthouse on a production build, throttled, and record LCP, CLS and INP.
2. Convert every img to Image with real dimensions and an honest sizes attribute.
3. Add priority to the one largest above-the-fold image, and nothing else.
4. Replace the font link tag with next/font and confirm CLS improves.
5. Move any third-party script to next/script with a lazy strategy.
6. Write .env.example and confirm .env*.local is gitignored.
7. Re-run Lighthouse and record the difference each change made.

Mini-Project · Making Addis Eats Fast

> **Tip What you build**
> A measured before-and-after. Every image through next/image with correct sizing and alt text, a self-hosted font, third-party scripts loaded lazily, and a clean environment setup with a committed example file. The deliverable is not the code — it is the documented improvement and the explanation of what caused it.

Requirements

- A Lighthouse run on a production build recorded before any change.
- Every image using next/image with width, height, sizes and meaningful alt text.
- priority on exactly one image, remote hosts listed, and next/font replacing any link tag.
- Third-party scripts on an appropriate next/script strategy.
- .env.example committed, .env*.local gitignored, no secret prefixed NEXT_PUBLIC.
- PERF.md with before and after numbers and what caused each change.

Check yourself

- Did LCP improve, and can you name the single change that did most of it?
- Does the page still jump as it loads? If so, what is not reserving space?

- Are you measuring npm run start, throttled — not npm run dev?
- Can a stranger clone your repository and know which variables to set?
- Is there anything in the browser bundle you would not want published?